

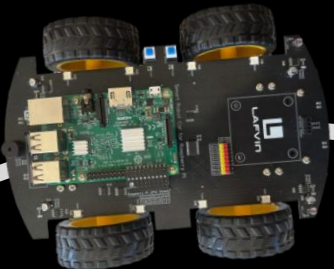
# Gruppe JoChBotics

Heidenheim Gran Prix, Programmieren 1 2026, Robert Luft



# Agenda

- Team Name
- Einsatz Clean Code
- Aufbau Algorithmus
- Entscheidungsfindung für den Algorithmus



# Team Name

- JoCh kommt von **J**onas und **C**hristoph
- Zusatz Botics kommt vom Wort **Ro**botics
- Roboter finden meist Einsatz in der Automatisierung
- Soll auf die Automation des Linienfolgens hinweisen

**JoChBotics**



# Einsatz Clean Code

```
if not sensor_left and sensor_mid and not sensor_right:  
    return MID
```

Verwendung von „not“ anstatt von „==False“.  
Dies stellt eine kleine Änderung dar, die aber zu einer merklich **besseren Lesbarkeit** führt.



# Einsatz Clean Code

```
def pos_sensor_over_line(sensor_orientation):  
    if sensor_orientation == "right":  
        sensor_value_right = linesensor_right.value  
        return sensor_value_right  
    elif sensor_orientation == "left":  
        sensor_value_left = linesensor_left.value  
        return sensor_value_left  
    else:  
        sensor_value_mid = linesensor_mid.value  
        return sensor_value_mid
```

Hier half die **Modularität** sehr weiter, da so in der Logik immer **dieselbe Funktion** genutzt werden konnte. Nur die gewünschte Sensor Position muss eingegeben werden.



# Einsatz Clean Code

```
FULL_LEFT = -1.0  
HALF_LEFT = -0.5  
MID = 0.0  
HALF_RIGHT = 0.5  
FULL_RIGHT = 1.0
```

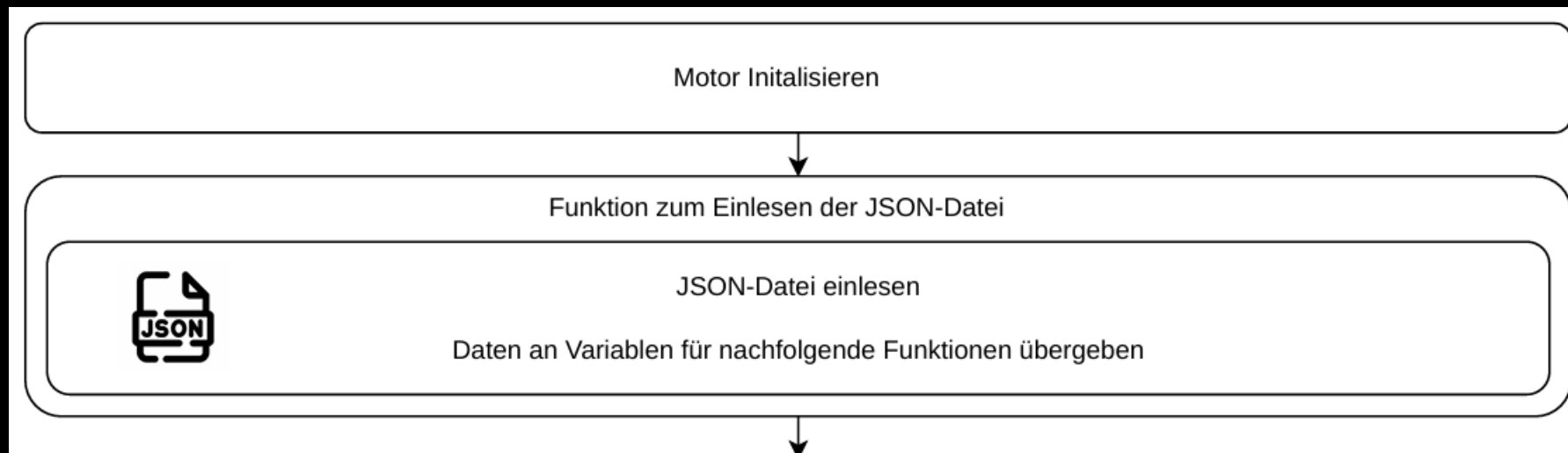
Die Deklaration von **Konstanten** und das damit verbundene GROSSSCHREIBEN half am Schluss vor allem, um variable Werte von Fixen zu unterscheiden.

Dies wurde auch zum Begrenzen der Motordrehzahl genutzt, da hier die 100% auch ein fixer Wert sind.



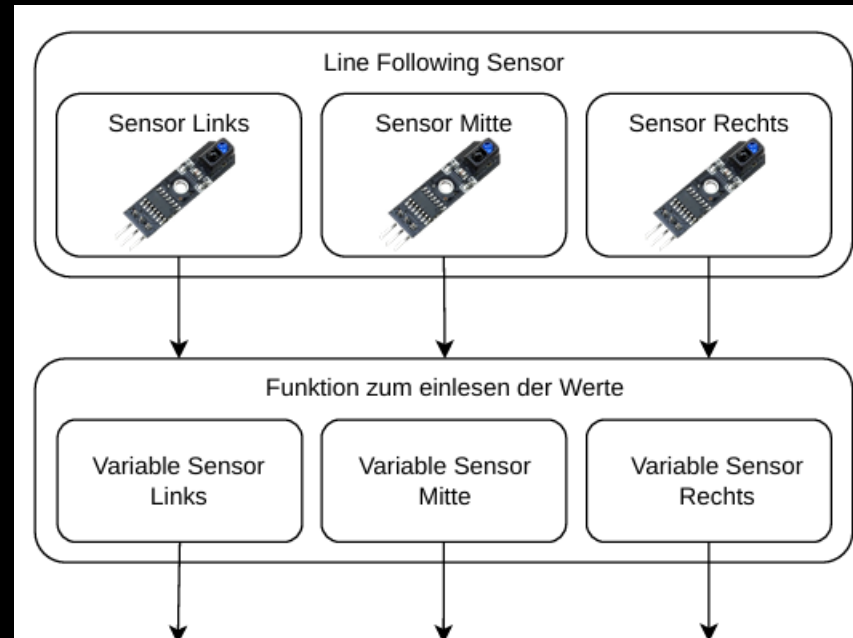
# Aufbau Algorithmus

Beim Aufruf von main.py:



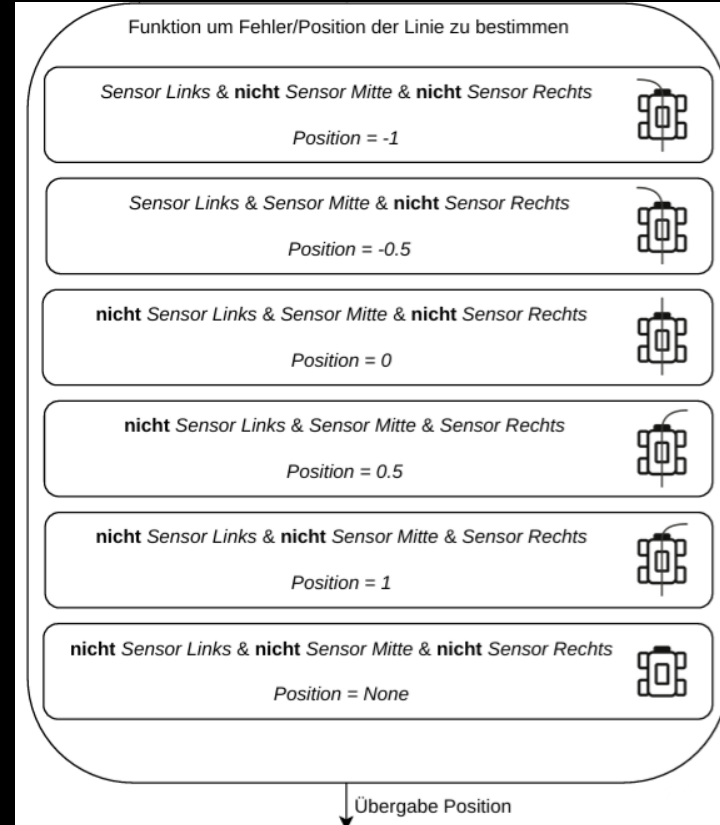
# Aufbau Algorithmus

Ab hier While True Schleife:

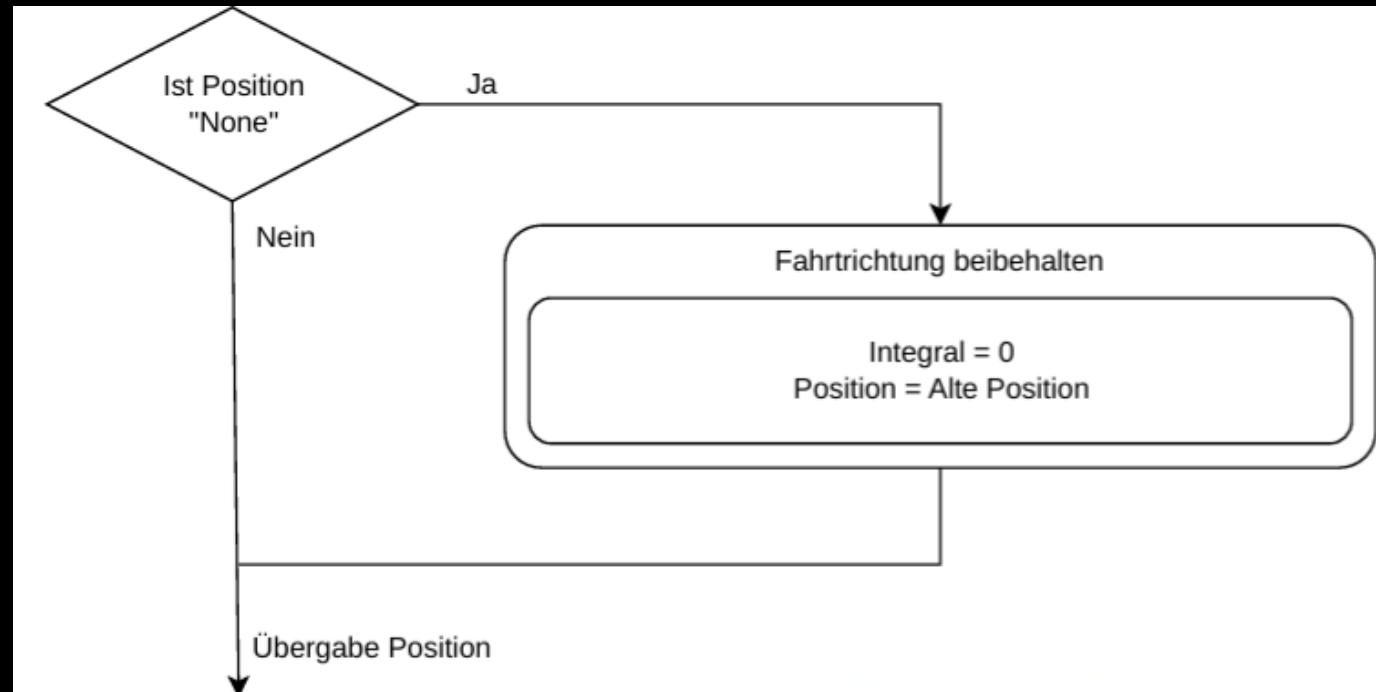




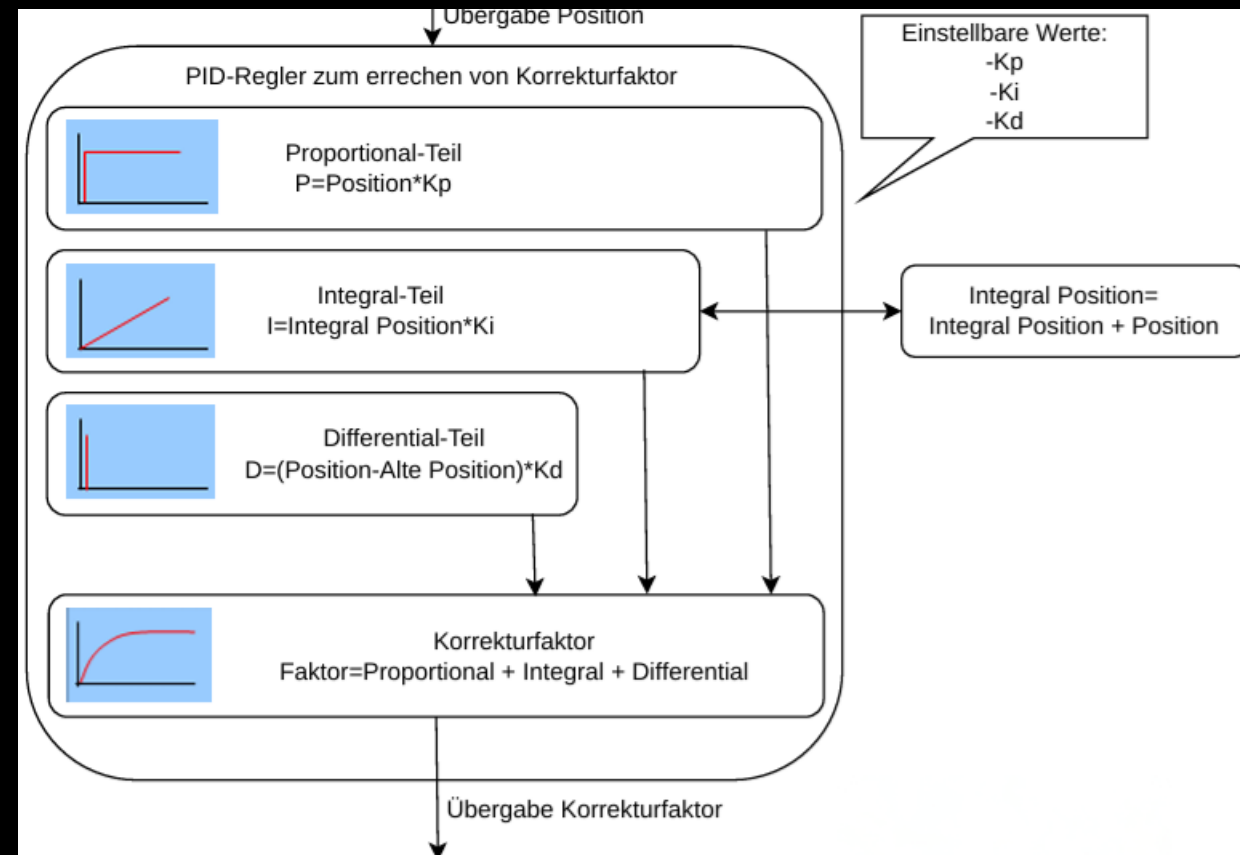
# Aufbau Algorithmus



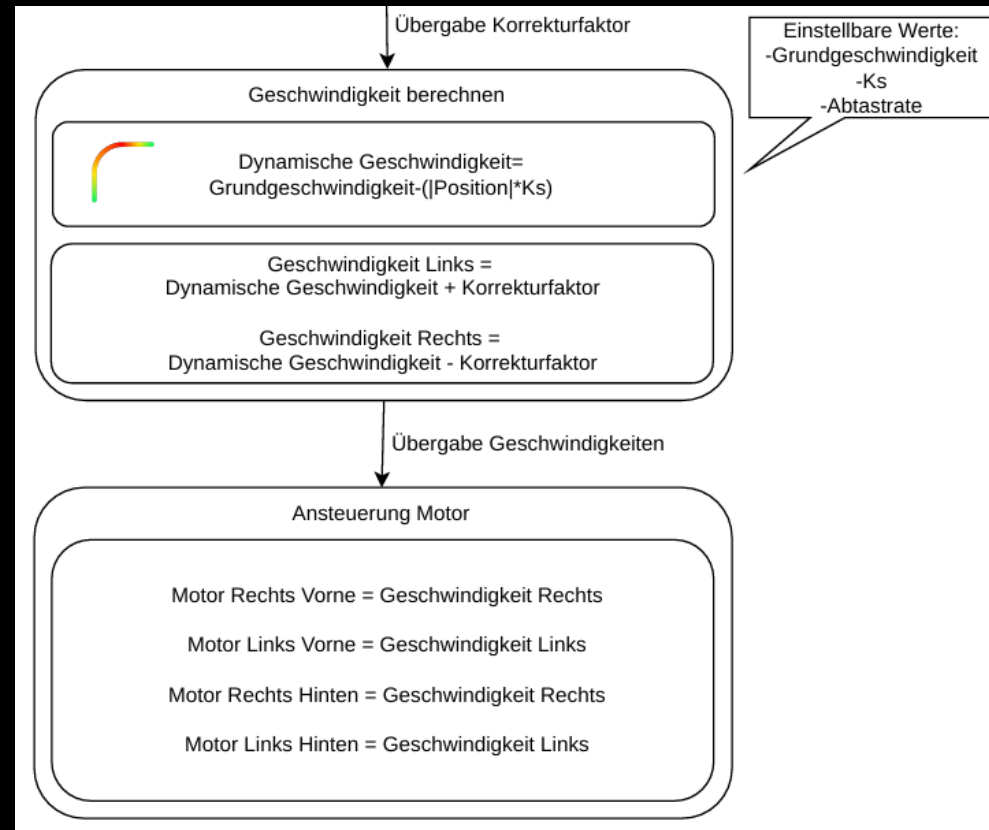
# Aufbau Algorithmus



# Aufbau Algorithmus



# Aufbau Algorithmus



# Entscheidungsfindung Algorithmus

- Zu Begin **Zweipunktregler**
  - Problem: „Schwingt“ sehr stark, ruppig
- Eigene Idee: **Werte dauerhaft** einlesen und Durschnitte bilden
- Im Internet auf den **PID-Regler** gestoßen und eine Idee, um die Durchschnitte verarbeiten zu können
  - Problem: Bilden der Durschnitte nicht performant genug



# Entscheidungsfindung Algorithmus

- Endgültig wurde ein **PID-Regler** mit deutlich **reduzierter Fehler-/Positionsbestimmung** in 0.5 Schritten von -1 bis 1 gewählt
- Die reduzierte Fehlererkennung wurde hierbei von einer anderen Gruppe entwickelt und von uns übernommen
- Ausblick: **Ziel** wäre es, das konstante Einlesen und Durchschnittsbilden performanter zu gestalten und erneut zu implementieren



# Danke für die Aufmerksamkeit

Nun lasst das Rennen beginnen

